Projet — API de gestion de garage automobile

Contexte

Un garage automobile souhaite digitaliser la gestion de ses véhicules et de leurs interventions. Vous devez créer une API HTTP en Go, port `8080`, avec persistence des données via une base de données SQLite (bibliothèque `modernc.org/sqlite`, sans dépendance C). En alternative, il est possible d'utiliser PostgreSQL, MySQL ou MariaDB.

Données

Véhicule : plaque d'immatriculation au format `AA-123-AA` (2 lettres, 3 chiffres, 2 lettres, identifiant unique), marque, modèle, année de mise en service, kilométrage, propriétaire (nom + téléphone).

Intervention : identifiant auto-généré, plaque du véhicule associé, date (`YYYY-MM-DD`), description des travaux, statut (`en_cours` , `terminee` , `annulee`).

Endpoints

Méthode	Route	Description
`GET`	`/vehicules`	Lister tous les véhicules
`POST`	`/vehicules`	Ajouter un véhicule
`GET`	`/vehicules/{plaque}`	Détails d'un véhicule
`DELETE`	`/vehicules/{plaque}`	Supprimer un véhicule (seulement si aucune intervention)
`POST`	`/vehicules/{plaque}/interventions`	Enregistrer une intervention
`GET`	`/vehicules/{plaque}/interventions`	Lister les interventions d'un véhicule
`GET`	`/stats`	Nombre de véhicules, nombre d'interventions par statut

Contraintes techniques

- Pas de framework HTTP externe (utiliser `net/http` de la bibliothèque standard).
- Base de données : `modernc.org/sqlite` (optionnel : PostgreSQL, MySQL ou MariaDB).
- Code organisé en packages séparés (point d'entrée, handlers, logique métier, modèle de données, erreurs).
- Routes paramétrées (`{plaque}`) gérées sans router externe.
- Réponses JSON avec tags de structure.
- Erreurs métier retournant un code HTTP pertinent et un message JSON.

Barème (15 points)

Critère	Points	Description
Modules et arborescence	/3	`go mod init` , structure en packages séparés (cmd/, internal/, etc.), `go.mod` et `go.sum` présents
Structs, composition et tags	/2	Structs avec composition, tags `json` , champs exportés/non exportés, récepteurs pointeur et valeur

Critère	Points	Description
Handlers HTTP et routage	/2	Tous les endpoints fonctionnels, routage paramétré, JSON en entrée et en sortie, codes HTTP appropriés
Gestion des erreurs	/3	Erreur custom avec interface `error`, sentinelles (`ErrNotFound`, etc.), utilisation de `errors.Is` / `errors.As`, propagation avec `%w`
Logique métier	/3	Validation des données, vérification des doublons, contrainte de suppression (pas d'intervention liée), calcul des stats
Qualité du code	/2	Code formaté (`go fmt`), pas de warning (`go vet`), style idiomatique Go (noms, conventions), commentaires doc sur les packages

Points bonus (5 points)

Critère	Points	Description
Tests unitaires	/2	Tests table-driven avec `testing`, au moins un fichier `_test.go` par package, utilisation de `t.Run` pour les sous-tests
Tests d'API avec httptest	/2	Tests des handlers avec `httptest.NewRecorder` et `httptest.NewRequest`, vérification des codes de réponse et du body JSON
Concurrence	/1	Au moins une goroutine avec channel ou `sync.WaitGroup` / `sync.Mutex` pour une opération concurrente (calcul de stats en parallèle)

Points de pénalité

Des points pourront être retirés sur le total en cas de manquements graves aux pratiques professionnelles suivantes :

- Le code ne compile pas ou génère des warnings avec `go vet`.
- Le serveur plante (panic non récupérée) lors de l'utilisation de l'API.
- Les erreurs sont ignorées avec `_` au lieu d'être traitées ou propagées.
- Un `panic` est utilisé comme mécanisme de retour d'erreur standard au lieu du pattern `(valeur, error)`.
- Le header `Content-Type: application/json` est absent des réponses JSON.
- Le body de la requête n'est pas validé ni limité en taille (vulnérabilité par déni de service).
- Aucun formatage (`go fmt`) n'a été appliqué au code rendu.